S.I.G.S.M.

Administración de Sistemas Operativos

Grupo ROKA

Rol	Apellido	Nombre	C.I	Email
Coordinador	Olivera	Benjamin	5.787.153-1	benjamin.olivera.contact@gmail.com
Sub-Coordinador	Alfonso	Agustin	5.724.747-1	agusalfonso486@gmail.com
Integrante 1	Katopulis	Thomas	5.775.049-8	katopulisthomas@gmail.com
Integrante 2	Rosas	Santiago	5.772.025-1	santrosasflores@gmail.com

Docente: Martinez, Santiago

Fecha de culminación

09/11/2026

TERCERA ENTREGA

I.S.B.O. **3MN**

ÍNDICE

ÍNDICE 2
Informe de Infraestructura del Servidor 3
Gestión de usuarios/grupos y firewall sobre CentOS Stream 9 3
1. Hardware necesario de los servidores 3
2. Costos de las diferentes formas de obtener el servidor 4
3. Sistema Operativo a utilizar 5
4. Requerimientos de un PC cliente de la aplicación 5
5. Paquetes necesarios para el funcionamiento del sistema 6
6. Manual de instalación básica del servidor (considerando el soporte técnico) 7
7. Relevamiento de incidentes de seguridad pasados sobre los paquetes a instalar 7
8. Descripción de vulnerabilidades en versiones anteriores del software 8
9. Manual de instalación de las herramientas (con estrategia de actualización) 9
7. Por último verificamos funcionamiento correcto de firewalld para con docker 10
10. Pruebas y evidencias de instalaciones y configuraciones correctas 11
11. Despliegue de la aplicación con Docker y Docker Compose 15
11.1 ¿Por qué usar Docker y Docker Compose? 15
11.2 Estructura del proyecto 17
11.3 Dockerfile del servicio web (PHP + Apache) 19
11.4 Archivo docker-compose.yaml 20
11.5 Archivo index.php de prueba de conexión 21
11.6 Puesta en marcha y comandos de uso 22
11.7 Configuración de firewalld y SELinux para los contenedores 22
11.8 Problemas encontrados y soluciones 23
11.9 Consideraciones de seguridad y estrategia de actualización 23
11.10 Pruebas y evidencias de Docker y Docker Compose 24
12. Administración del árbol de directorios y comandos del sistema operativo 28
12.1 Ubicación del proyecto en el servidor 28
12.2 Árbol de directorios real del proyecto 29
12.3 Rutas absolutas y relativas en Docker Compose 30
12.4 Configuración de acceso remoto al usuario de MySQL desde Docker 31
13. Referencias técnicas y fecha de revisión 31

Informe de Infraestructura del Servidor

Gestión de usuarios/grupos y firewall sobre CentOS Stream 9

---

1. Hardware necesario de los servidores

Dado que el servidor aloja una aplicación propia con scripts de gestión de firewall y de usuarios/grupos (carga baja-media, uso académico/piloto, un único servidor físico/virtual en la PC de un participante), el hardware recomendado es modesto:

Recurso	Mínimo	Recomendado
CPU	1 vCPU / núcleo	2 vCPU / núcleos
RAM	2 GB	4–8 GB
Disco	20 GB (SSD preferido)	40–80 GB SSD
Red	Ethernet 100 Mbps, IP fija en la red local	Ethernet 1 Gbps, IP fija + acceso remoto vía SSH
Alimentación	—	UPS recomendado (al alojarse en una PC doméstica, ante cortes de luz)

Justificación: CentOS Stream 9 en modo servidor mínimo (sin entorno gráfico) funciona cómodamente con estos valores. La aplicación en sí (scripts bash/python de administración) no demanda recursos intensivos; el margen adicional se reserva para el propio sistema operativo, logs, actualizaciones y la virtualización (VirtualBox) sobre el host físico.

---

2. Costos de las diferentes formas de obtener el servidor

Modalidad	Costo aproximado	Ventajas	Desventajas
Propio (PC de un participante)	Costo hundido del hardware ya existente + electricidad (bajo, unos pocos USD/mes)	Sin costo mensual, control total, ideal para pruebas académicas	Depende de la disponibilidad de esa PC/conexión; sin SLA; riesgo de corte de energía o de internet
VM contratada (VPS: DigitalOcean, Linode, Vultr, AWS Lightsail, etc.)	Desde ~USD 5–20/mes (1–2 vCPU, 1–4 GB RAM)	Disponibilidad alta, IP pública fija, sin depender de hardware propio	Costo recurrente, requiere tarjeta/cuenta, menos control físico
Bare metal contratado (servidor dedicado)	Desde ~USD 50–150+/mes	Máximo rendimiento, sin virtualización, control total del hardware	Costo elevado, sobredimensionado para este proyecto
VM propia (VirtualBox sobre PC propia)	Igual que "propio"	Permite practicar instalación, snapshots, fácil de resetear	Rendimiento limitado por el host, no accesible públicamente sin configurar NAT/port forwarding

Conclusión para este proyecto: la opción elegida (servidor propio, virtualizado en VirtualBox, alojado en la PC de un participante) es la de menor costo y la más adecuada para un entorno académico/de desarrollo, aceptando como contrapartida la falta de alta disponibilidad.



3. Sistema Operativo a utilizar

Aclaración importante: no existe "CentOS 9" como tal. Desde el fin de CentOS Linux 8, el proyecto continúa como **CentOS Stream 9**, que es la distribución que corresponde usar aquí.

- **Soporte técnico:** CentOS Stream no tiene soporte comercial propio; el soporte es **comunitario** (listas de correo, foros oficiales, documentación oficial de Red Hat/CentOS).
- **Alternativas si se necesitara soporte pagado:** migrar a **RHEL** (con suscripción de Red Hat, incluye soporte oficial) o a un clon compatible con soporte de terceros (Alma Linux, Rocky Linux, ambos con soporte comunitario similar a CentOS Stream, y con opciones de soporte empresarial de terceros).
- **Ciclo de vida:** CentOS Stream 9 recibe actualizaciones hasta mayo de 2027, alineado al ciclo de RHEL 9.

Para este proyecto, al ser un entorno propio sin necesidad de SLA formal, el soporte comunitario es suficiente.

---

4. Requerimientos de un PC cliente de la aplicación

Recurso	Mínimo
CPU	Cualquier CPU de los últimos 10 años
RAM	4 GB
Sistema operativo	Windows, Linux o macOS (cualquiera con cliente SSH y navegador)
Software	Cliente SSH (o Visual Studio Code con extensión Remote-SSH), navegador web actualizado si la app tiene interfaz web
Red	Conexión a la misma red local que el servidor, o acceso remoto configurado (VPN/port forwarding)

No se requiere hardware especial: el cliente solo necesita poder conectarse por SSH (para administración) y/o consumir la aplicación (web, API, etc.).

---

5. Paquetes necesarios para el funcionamiento del sistema

Paquete	Función	Criterio de elección
`openssh-server`	Acceso remoto seguro (edición de scripts desde VS Code)	Estándar de facto, mantenido activamente, cifrado robusto, última versión estable en repos oficiales
`firewalld`	Gestión de firewall (backend nftables)	Paquete oficial de CentOS Stream, integración nativa con `systemd`, sintaxis por zonas más simple que `iptables` puro
`sudo`	Elevación de privilegios controlada	Imprescindible para administración segura sin usar root directo
`shadow-utils` (useradd/groupadd/usermod, etc.)	Gestión de usuarios y grupos	Viene en la instalación base, es el estándar POSIX en RHEL/CentOS
`policycoreutils` / `setroubleshoot`	Gestión de SELinux	CentOS trae SELinux activo por defecto; necesario para diagnosticar bloqueos de los scripts
`nftables` (dependencia de firewalld)	Motor de filtrado de paquetes	Reemplazo moderno de iptables, más eficiente
`chrony`	Sincronización horaria	Necesario para logs y auditoría coherentes (SSH, firewall)

`git`, `nano`/`vim`	Edición y control de versiones de los scripts	Herramientas livianas, ya presentes en repos base

Criterios generales de elección: paquetes oficiales de los repos de CentOS Stream/EPEL (no de terceros no verificados), con mantenimiento activo, huella mínima, y buen historial de seguridad.

---

6. Manual de instalación básica del servidor (considerando el soporte técnico)

1. **Crear la VM en VirtualBox:** Nueva VM → tipo "Red Hat (64-bit)" → asignar 2 vCPU, 4 GB RAM, 40 GB disco (VDI, reservado dinámicamente).
2. **Configurar red:** Adaptador en modo **Puente** (para obtener IP en la red local y permitir SSH desde otras PCs) o NAT con reenvío del puerto 22.
3. **Montar la ISO** de CentOS Stream 9 (descargada del sitio oficial) y arrancar la VM.
4. **Instalación:** elegir "Minimal Install" (sin entorno gráfico, reduce superficie de ataque), configurar particionado, zona horaria, usuario administrador y habilitar red al inicio.
5. **Primer arranque:** `sudo dnf update -y` para dejar el sistema al día.
6. **Registrar la IP fija** para que la conexión SSH desde VS Code sea estable.
7. **Soporte técnico:** ante problemas de instalación, consultar la documentación oficial de CentOS Stream, el foro `discussion.centos.org`, y la wiki de Red Hat. No hay línea de soporte pagada salvo migración a RHEL.

---

7. Relevamiento de incidentes de seguridad pasados sobre los paquetes a instalar

Paquete	Incidente relevante
OpenSSH	CVE-2024-6387 ("regreSSHit"), condición de carrera en `sshd` que podía permitir ejecución remota de código en versiones 8.5p1–9.7p1 (corregida en 9.8p1, 2024).

sudo	CVE-2021-3156 ("Baron Samedit"), desbordamiento de buffer basado en heap explotable por usuarios sin privilegios para escalar a root (corregido en sudo 1.9.5p2).
firewalld / nftables	Diversas CVEs menores de denegación de servicio y errores de parseo de reglas a lo largo de los años; ninguna crítica reciente de ejecución remota.
glibc / systemd (dependencias base)	Historial de CVEs de escalamiento de privilegios local, mitigadas con actualizaciones regulares del sistema.

Este relevamiento confirma que **incluso paquetes estándar y confiables han tenido vulnerabilidades críticas**, lo que justifica una política de actualizaciones periódicas.

---

8. Descripción de vulnerabilidades en versiones anteriores del software

- **OpenSSH < 9.8p1:** la vulnerabilidad "regreSSHit" era una condición de carrera en el manejo de señales del `sshd`, reintroducida por una regresión respecto a una corrección de 2006; en condiciones específicas permitía comprometer el proceso `sshd` (ejecutándose como root) de forma remota y sin autenticación previa.
- **sudo < 1.9.5p2:** el fallo "Baron Samedit" estaba en el parseo de argumentos de línea de comandos de `sudoedit`/`sudo -e`; un usuario local sin privilegios especiales podía desencadenar un desbordamiento de heap y obtener privilegios de root.
- **iptables/nftables (versiones antiguas):** se registraron fallos que permitían, en ciertos escenarios, bypass de reglas o denegación de servicio del propio proceso de filtrado.

Lección aplicada al proyecto: instalar siempre la **última versión estable disponible en los repositorios oficiales** de CentOS Stream 9 (que ya incorpora estas correcciones) y mantener una rutina de parches.



9. Manual de instalación de las herramientas (con estrategia de actualización)

```
# Actualizar el sistema
sudo dnf update -y

# Instalar y habilitar firewalld
sudo dnf install -y firewalld
sudo systemctl enable --now firewalld

# Instalar y habilitar SSH
sudo dnf install -y openssh-server
sudo systemctl enable --now sshd
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --reload

# Herramientas de gestión de usuarios y grupos
rpm -q shadow-utils

# sudo y utilidades adicionales
sudo dnf install -y sudo policycoreutils policycoreutils-python-utils git nano chrony
sudo systemctl enable --now chronyd

# --- Docker y Docker Compose ---

# 1. Plugin para gestionar repositorios
sudo dnf install -y dnf-plugins-core
```



2. Agregar el repositorio oficial de Docker para CentOS Stream 9

sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo

3. Instalar Docker Engine, CLI, containerd, Buildx y Compose

sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

4. Habilitar e iniciar el servicio

sudo systemctl enable --now docker

5. Permitir usar Docker sin sudo (cierra sesión y vuelve a entrar para que aplique)

sudo usermod -aG docker $USER

6. Verificar la instalación

docker --version

docker compose version

sudo docker run --rm hello-world

7. Por último verificamos funcionamiento correcto de firewalld para con docker

sudo firewall-cmd --permanent --zone=trusted --add-interface=docker0

sudo firewall-cmd --reload

sudo systemctl restart docker

Estrategia de actualizaciones:

- Actualizaciones manuales periódicas con `sudo dnf update -y`, revisando el changelog antes de aplicar en el servidor (para evitar romper la configuración de firewall/SSH).
- Alternativa: `dnf-automatic` con `apply_updates = yes` solo para parches de seguridad (`--security`), dejando las actualizaciones mayores para aplicación manual y controlada.
- Registrar cada actualización aplicada (fecha, paquetes, versión) como parte de la evidencia del punto 10.

---

10. Pruebas y evidencias de instalaciones y configuraciones correctas

A realizar y documentar por los estudiantes (capturas de pantalla + salida de comandos):

1. **SSH:** `systemctl status sshd` (activo) + conexión exitosa desde VS Code (Remote-SSH) mostrando la sesión abierta.

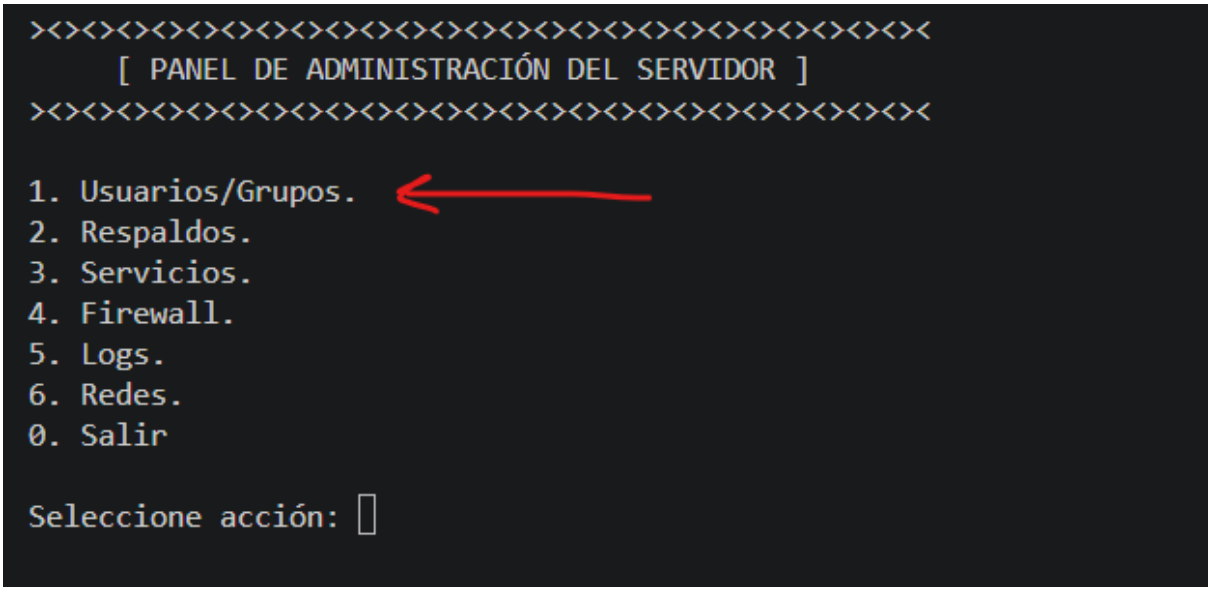
```
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-11-06 00:34:05 -03; 11h ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 852 (sshd)
      Tasks: 1 (limit: 23109)
     Memory: 7.7M (peak: 8.9M)
        CPU: 191ms
     CGroup: /system.slice/sshd.service
             └─852 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

2. **Firewall:** `sudo firewall-cmd --list-all` mostrando el servicio SSH habilitado y las zonas configuradas; prueba de conexión bloqueada a un puerto no habilitado.

```
[benjamin@192 mi-proyecto]$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s3
  sources:
  services: cockpit dhcpv6-client ssh
  ports: 8080/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

3. **Usuarios y grupos:** ejecución del script de alta de usuario/grupo, verificación con `id <usuario>` y `getent group <grupo>`.

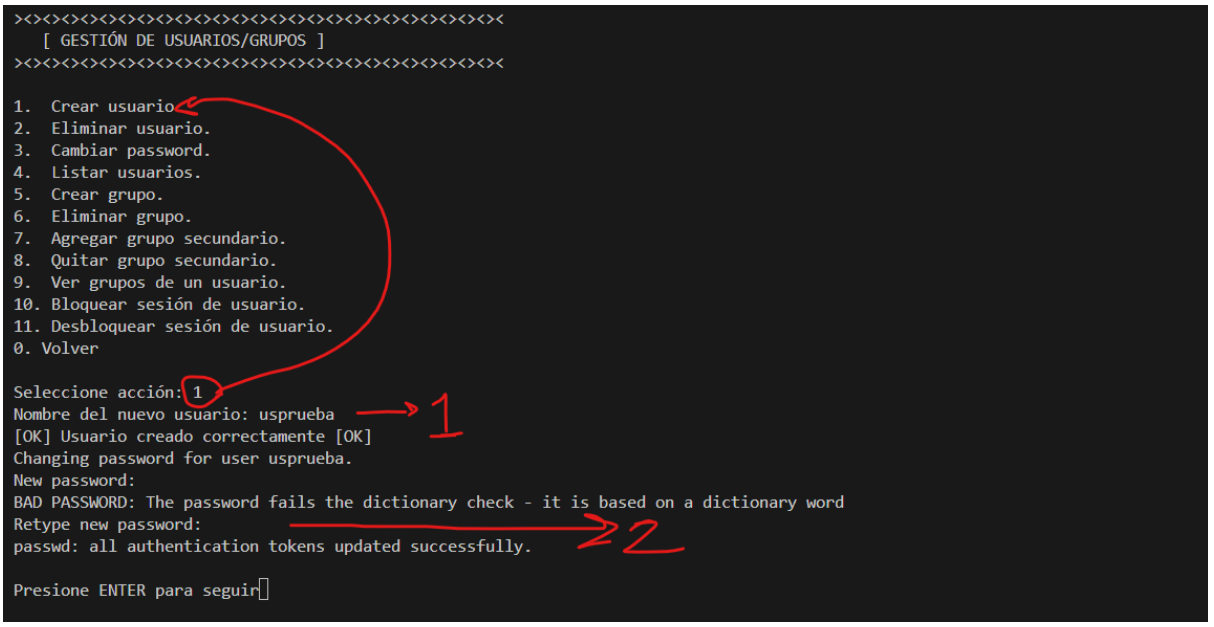
   Primero selecciono la opción requerida:



   Luego creo un usuario de prueba:

   1- le pongo nombre: usprueba

   2- le pongo contraseña: prueba1234



Por último, verifico que el usuario existe efectivamente:

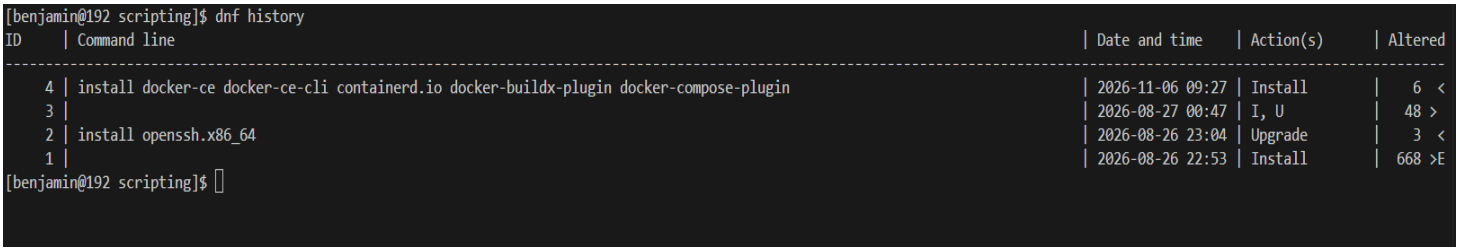
1- uso la opción 4 que lista los usuarios mediante el siguiente funcionamiento:

```bash
listarUsuarios() {
    local usuario pass uid guid comment home shell
    echo "Usuarios con UID mayor o igual a 1000: "
    echo
## "IFS=:" es un separador, guarda en cada variable lo que capta read en cada campo delimitado por los ":"
    while IFS=: read -r usuario pass uid guid comment home shell; do
        if [[ $uid -ge 1000 && $usuario != "nobody" ]]; then
            echo "Usuario: $usuario"
            echo "UID:     $uid"
            echo "Home:    $home"
            echo "Shell:   $shell"
            echo "-----------------------------------"
        fi
    done < /etc/passwd
}
```



Efectivamente existe y tiene su propio home.

4. **Actualizaciones:** salida de `dnf history` mostrando las transacciones de actualización aplicadas.



5. **Persistencia:** reinicio de la VM y verificación de que `sshd` y `firewalld` arrancan automáticamente (`systemctl is-enabled`).

verificamos firewalld:

```
[benjamin@192 scripting]$ sudo systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
     Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-11-06 00:34:05 -03; 12h ago
       Docs: man:firewalld(1)
   Main PID: 763 (firewalld)
      Tasks: 4 (limit: 23109)
     Memory: 51.0M (peak: 59.2M)
        CPU: 3.836s
     CGroup: /system.slice/firewalld.service
             └─763 /usr/bin/python3 -s /usr/sbin/firewalld --nofork --nopid
```

verificamos sshd:

```
[benjamin@192 scripting]$ sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
     Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-11-06 00:34:05 -03; 12h ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 852 (sshd)
      Tasks: 1 (limit: 23109)
     Memory: 7.7M (peak: 8.9M)
        CPU: 191ms
     CGroup: /system.slice/sshd.service
             └─852 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Como se puede ver, el preset está configurado en enabled.

11. Despliegue de la aplicación con Docker y Docker Compose

Además de la configuración del servidor descrita en los puntos anteriores, la aplicación web (PHP + Apache) y su base de datos (MySQL) se despliegan mediante **contenedores Docker**, administrados con **Docker Compose**. En este punto se documenta la estructura del proyecto, los archivos de configuración utilizados, la puesta en marcha, los problemas encontrados y las pruebas realizadas. La instalación de Docker y del plugin de Compose ya quedó detallada en el punto 9.

11.1 ¿Por qué usar Docker y Docker Compose?

Docker permite empaquetar cada servicio junto con sus dependencias en un contenedor aislado, de forma que la aplicación funciona igual en cualquier equipo y no es necesario instalar PHP, Apache ni MySQL directamente sobre el CentOS Stream 9 del servidor. **Docker Compose** complementa a Docker describiendo, en un único archivo (docker-compose.yaml), todos los servicios que forman la aplicación y cómo se relacionan entre sí.

Motivo principal: es más práctico levantar los dos contenedores en un solo enter. Sin Compose habría que repetir manualmente, y en el orden correcto, los siguientes pasos:

1. Construir la imagen del servidor web (docker build).
2. Crear una red para que el servidor web y la base de datos puedan comunicarse (docker network create).
3. Crear y arrancar el contenedor de MySQL con sus variables de entorno, puertos y volúmenes (docker run con una lista larga de parámetros).
4. Crear y arrancar el contenedor web con su volumen, su puerto y respetando que la base de datos esté arriba primero (otro docker run).

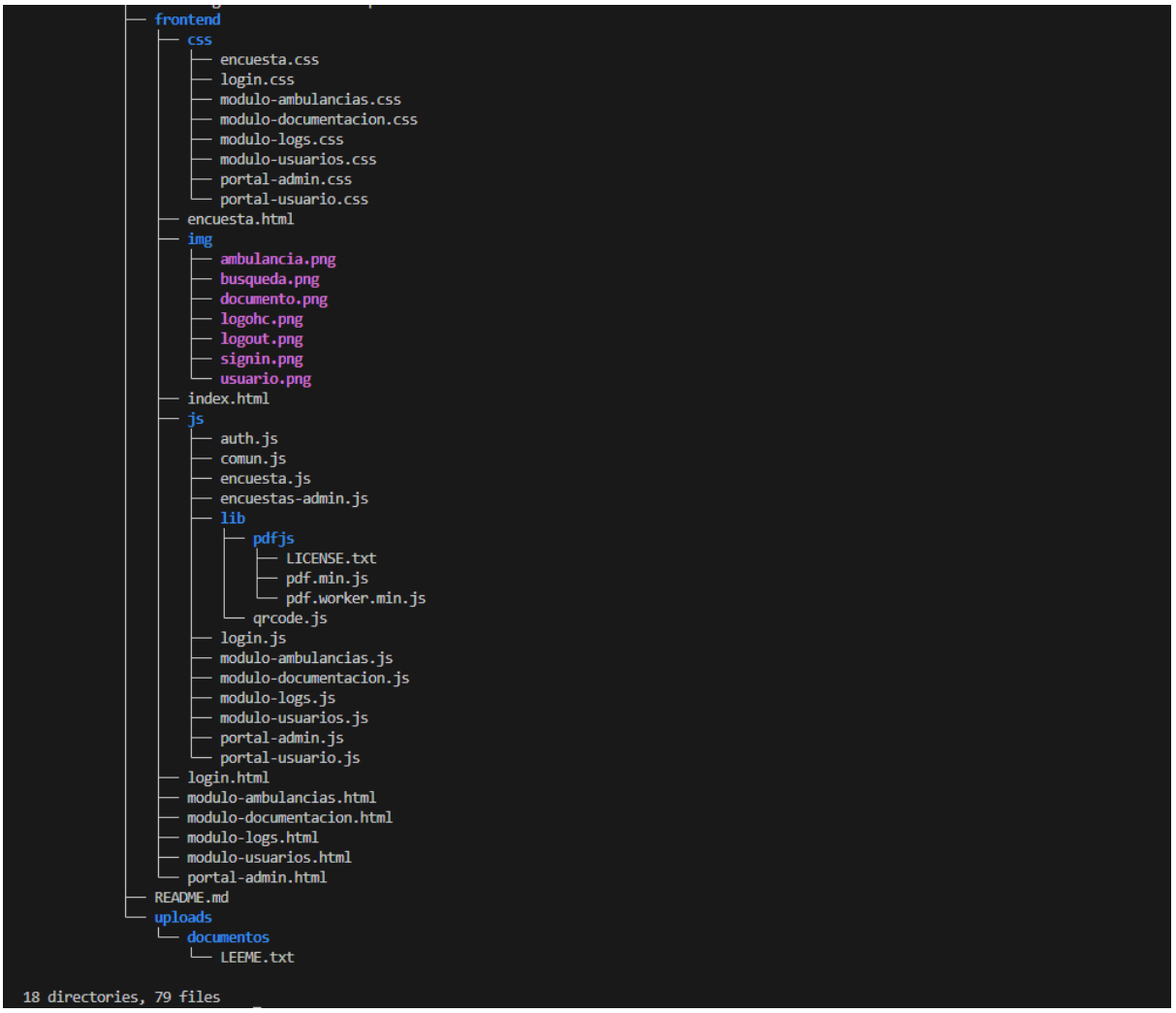
Con Docker Compose todo eso se resume en un único comando, **docker compose up -d**, y de la misma forma se detiene todo con **docker compose down**. Otras ventajas que aporta al proyecto:

- **Reproducibilidad:** la configuración queda escrita en un archivo que se puede versionar con git (ya instalado en el servidor) y volver a desplegar idéntica en otro equipo.
- **Comunicación automática:** Compose crea una red interna donde el servidor web alcanza a la base de datos simplemente por el nombre del servicio (db).
- **Orden y recuperación:** depends_on controla el orden de arranque y restart vuelve a levantar los contenedores si se caen o si se reinicia el servidor.
- **Aislamiento:** los servicios no ensucian el sistema operativo base y se pueden eliminar sin dejar rastros.

11.2 Estructura del proyecto

Todos los archivos se organizan dentro de la carpeta mi-proyecto, en el directorio personal del usuario administrador:

```
[benjamin@192 mi-proyecto]$ tree
.
├── docker-compose.yaml
└── php-apache
    ├── Dockerfile
    └── src
        ├── index.php
        └── sigsm
            ├── api
            │   ├── categorias.php
            │   ├── documentos.php
            │   ├── encuestas.php
            │   ├── index.php
            │   ├── login.php
            │   ├── logout.php
            │   ├── logs.php
            │   ├── pacientes.php
            │   ├── pdf.php
            │   ├── sesion.php
            │   ├── traslados.php
            │   ├── usuarios.php
            │   └── vehiculos.php
            ├── backend
            │   ├── config
            │   │   └── Database.php
            │   ├── database
            │   │   ├── 01_sigsm.sql
            │   │   └── 02_usuarios_bd.sql
            │   ├── models
            │   │   ├── Categoria.php
            │   │   ├── Documento.php
            │   │   ├── Encuesta.php
            │   │   ├── Item.php
            │   │   ├── Log.php
            │   │   ├── Paciente.php
            │   │   ├── Peticion.php
            │   │   ├── Respuesta.php
            │   │   ├── Usuario.php
            │   │   └── Vehiculo.php
            │   └── negocio
            │       ├── Autorizacion.php
            │       ├── ReglaNegocioException.php
            │       ├── ReglasTraslado.php
            │       ├── ServicioCategoria.php
            │       ├── ServicioDocumento.php
            │       ├── ServicioEncuesta.php
            │       ├── ServicioTraslado.php
            │       └── ServicioUsuario.php
            └── docs
                ├── DERFinal.png
                └── ProgramaciónFullStack.pdf
```



- **docker-compose.yaml:** archivo principal; describe los dos contenedores, sus puertos, volúmenes y variables.
- **php-apache/Dockerfile:** receta para construir la imagen del servidor web.
- **php-apache/src:** carpeta con el código de la aplicación. Se monta dentro del contenedor, por lo que los cambios en index.php se ven al recargar el navegador, sin reconstruir la imagen.
- **SQL/DB.sql:** script que MySQL ejecuta automáticamente la primera vez que se crea la base de datos.

11.3 Dockerfile del servicio web (PHP + Apache)

El servidor web se construye a partir de la imagen oficial de PHP con Apache, a la que se le agrega la extensión mysqli para poder conectarse a MySQL:

```
# Imagen oficial de PHP con Apache (Debian bookworm)
FROM php:8.2-apache

# Evitar preguntas interactivas durante la instalación de paquetes
ENV DEBIAN_FRONTEND=noninteractive

# Actualizar índices e instalar herramientas útiles (opcional), limpiando después
RUN apt-get update \
    && apt-get install -y --no-install-recommends vim \
    && rm -rf /var/lib/apt/lists/*

# Extensión para conectar PHP con MySQL
RUN docker-php-ext-install mysqli

WORKDIR /var/www/html

EXPOSE 80

# Apache ya arranca por defecto con el CMD de la imagen base
```

- **FROM:** imagen base oficial php:8.2-apache, basada en Debian 12 (bookworm), mantenida y con parches de seguridad al día.
- **RUN apt-get ...:** actualiza los índices e instala vim en un solo paso y limpia la caché para que la imagen quede liviana. Se evitó apt-get upgrade, que no es necesario y alarga la construcción.
- **docker-php-ext-install mysqli:** habilita la extensión que usa index.php para hablar con la base de datos.
- **WORKDIR y EXPOSE:** fijan el directorio de trabajo (/var/www/html) y documentan el puerto 80 de Apache.

11.4 Archivo docker-compose.yaml

Este archivo define los dos servicios de la aplicación. Es importante respetar la indentación (2 espacios): las claves de cada servicio deben quedar al mismo nivel que build o image, no dentro de ellas.

```yaml
▷Run All Services
services:
  ▷Run Service
  web:
    build:
      context: /home/benjamin/mi-proyecto/php-apache
      dockerfile: Dockerfile
    container_name: servidor_web
    volumes:
      - /home/benjamin/mi-proyecto/php-apache/src:/var/www/html
    ports:
      - "8080:80"
    restart: always
    depends_on:
      - db

  ▷Run Service
  db:
    image: mysql:8.0  # Usando la imagen oficial de MySQL
    container_name: base_datos_sql
    command: --default-authentication-plugin=mysql_native_password
    environment:
      MYSQL_ROOT_PASSWORD: root1234
      MYSQL_DATABASE: mi_base
      MYSQL_USER: usuario
      MYSQL_PASSWORD: user1234
    ports:
      - "6033:3306"
    volumes:
      - datosDB:/var/lib/mysql
      - /home/benjamin/mi-proyecto/SQL/DB.sql:/docker-entrypoint-initdb.d/init.sql  # Carga automática del script
    restart: unless-stopped
volumes:
  datosDB:
    external: true ## pq ya existe el volumen
```

- **web:** se construye con el Dockerfile del punto anterior, publica el puerto 8080 del servidor hacia el 80 del contenedor y monta la carpeta src como raíz del sitio. depends_on indica que debe arrancar después de db y restart: always lo mantiene siempre activo.
- **db:** usa la imagen oficial mysql:8.0 y crea automáticamente la base mi_base y el usuario de la aplicación a partir de las variables de entorno. Al montar DB.sql en docker-entrypoint-initdb.d, el script se ejecuta solo la primera vez que se crea la base.
- **volumes (datosDB):** volumen con nombre administrado por Docker donde MySQL guarda sus datos. Gracias a él la información se conserva aunque los contenedores se eliminen y se vuelvan a crear. Debe declararse también al final del archivo, de lo contrario Compose devuelve error.

- **Sufijo :z:** indica a Docker que reetiquete los archivos montados para SELinux, que en CentOS Stream 9 viene activo por defecto (ver punto 11.7).

Los puertos publicados hacia el servidor son los siguientes:

Servicio	Puerto del servidor	Puerto del contenedor	Uso
web (servidor_web)	8080	80	Acceso a la aplicación desde el navegador
db (base_datos_sql)	6033	3306	Administración de la base desde el servidor

Dentro de la red de Compose la aplicación no usa el puerto 6033: se conecta a la base con el nombre del servicio (db) y el puerto interno 3306.

11.5 Archivo index.php de prueba de conexión

Para comprobar que el servidor web y la base de datos se comunican correctamente se utiliza el siguiente index.php, ubicado en php-apache/src:

```php
<?php
$host = "db";
$user = "usuario";
$pass = "user1234";
$dbname = "mi_base";

$conn = new mysqli($host, $user, $pass, $dbname);

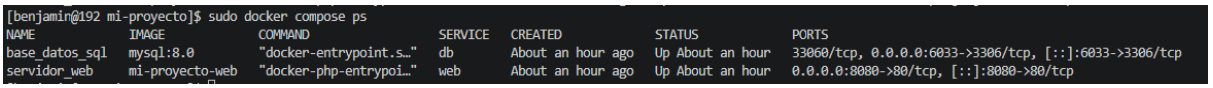
if ($conn->connect_error) {
    die("Error de conexión: " . $conn->connect_error);
}

echo "Conexión exitosa a la base de datos <strong>$dbname</strong>";
$conn->close();
```

El script intenta abrir una conexión con las mismas credenciales definidas en el compose. Si todo está correcto, el navegador muestra el mensaje de conexión exitosa; de lo contrario, muestra el motivo del error.

11.6 Puesta en marcha y comandos de uso

Desde la carpeta del proyecto, se construye la imagen y se levantan ambos contenedores en segundo plano con un solo comando, y luego se verifica su estado:



La aplicación queda disponible en http://localhost:8080 (o en http://IP_DEL_SERVIDOR:8080 desde otra PC de la red). La primera vez, MySQL tarda unos segundos en inicializarse: cuando en los logs de db aparece el mensaje ready for connections, la base ya está lista. Los comandos más utilizados son:

Comando	Función
docker compose up -d	Construye (si hace falta) y levanta los contenedores en segundo plano
docker compose ps	Muestra el estado de los contenedores del proyecto
docker compose logs web / db	Muestra los registros de cada servicio, útil para diagnosticar errores
docker compose build --no-cache	Reconstruye la imagen del servidor web desde cero
docker compose down	Detiene y elimina los contenedores; los datos se conservan en el volumen
docker compose down -v	Igual que el anterior, pero también elimina el volumen: se pierden los datos y DB.sql vuelve a ejecutarse

11.7 Configuración de firewalld y SELinux para los contenedores

Como el servidor tiene firewalld y SELinux activos (puntos 5 y 9), hay que contemplarlos para que los contenedores funcionen sin desactivar ninguna de las dos protecciones:

- **firewalld:** la interfaz docker0 se agrega a la zona trusted (paso 7 del punto 9) para que los contenedores tengan salida a internet, por ejemplo durante apt-get update en la construcción de la imagen. Además, para acceder a la aplicación desde otra PC de la red se habilita el puerto 8080.

- **SELinux:** al montar carpetas del directorio personal dentro de un contenedor, SELinux puede bloquear el acceso (errores 403 Forbidden o Permission denied). Se resuelve agregando :z al final de cada bind mount, tal como se muestra en el compose.

```
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload
```

El puerto 6033 de MySQL no se habilita en el firewall, ya que solo se utiliza para administración desde el propio servidor. Como medida adicional, podría publicarse únicamente en la interfaz local (127.0.0.1:6033:3306) o eliminarse, porque la aplicación accede a la base por la red interna de Compose.

11.8 Problemas encontrados y soluciones

Durante el despliegue aparecieron tres errores que se documentan a continuación, junto con su causa y la corrección aplicada:

Error	Causa	Solución
services.web.build additional properties 'container_name', 'depends_on', 'ports', 'restart', 'volumes' not allowed	Error de indentación: las claves del servicio web quedaron dentro de build.	Alinear container_name, volumes, ports, restart y depends_on al mismo nivel que build (2 espacios por nivel).
service "db" refers to undefined volume datosDB: invalid compose project	El volumen con nombre datosDB se usaba en el servicio db pero no estaba declarado.	Agregar la sección volumes: con datosDB: al final del archivo, sin indentación.
failed to solve: process "/bin/sh -c apt-get update && apt-get upgrade -y" did not complete successfully: exit code: 100	apt-get no pudo descargar los repositorios durante la construcción: imagen base antigua (bullseye) y/o falta de salida a internet desde el contenedor.	Usar la imagen php:8.2-apache (bookworm), quitar apt-get upgrade y agregar docker0 a la zona trusted de firewalld. Para diagnosticar: docker compose build --no-cache --progress=plain web.

11.9 Consideraciones de seguridad y estrategia de actualización

- **Grupo docker:** pertenecer al grupo docker equivale a tener privilegios de root en el equipo, por lo que solo debe asignarse a usuarios administradores.

- **Credenciales:** las contraseñas están escritas en texto plano en el compose, lo cual es aceptable en un entorno académico/piloto. En un entorno real conviene moverlas a un archivo .env fuera del repositorio o usar secretos de Docker.

- **Actualización de Docker:** como el repositorio oficial de Docker queda registrado en dnf, el sudo dnf update -y del punto 9 también actualiza el motor. Luego de actualizar, se verifica con docker compose ps que los contenedores sigan arriba.

- **Actualización de imágenes:** se ejecutan docker compose pull (descarga la última versión de mysql:8.0), docker compose build --pull (reconstruye el servidor web sobre la última imagen base) y finalmente docker compose up -d para recrear los contenedores. Cada actualización se registra como evidencia, igual que en el punto 9.

11.10 Pruebas y evidencias de Docker y Docker Compose

A realizar y documentar por los estudiantes (capturas de pantalla + salida de comandos):

1. **Instalación de Docker:** docker --version, docker compose version y sudo docker run --rm hello-world, mostrando que el motor y el plugin de Compose están instalados y funcionan.

```
[benjamin@192 mi-proyecto]$ sudo docker --version
Docker version 29.9.3, build 5457199
```

```
[benjamin@192 mi-proyecto]$ sudo docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

2. **Servicio activo y persistente:** systemctl status docker (activo) y systemctl is-enabled docker (enabled), para comprobar que Docker arranca solo al reiniciar el servidor.

```
[benjamin@192 mi-proyecto]$ systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
     Active: active (running) since Fri 2026-11-06 15:55:10 -03; 1min 39s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 38039 (dockerd)
      Tasks: 41
     Memory: 48.9M (peak: 49.2M)
        CPU: 451ms
     CGroup: /system.slice/docker.service
             ├─38039 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
             ├─38376 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 6033 -container-ip 172.18.0.2 -container-port 3306 -use-listen-fd
             ├─38382 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 6033 -container-ip 172.18.0.2 -container-port 3306 -use-listen-fd
             ├─38411 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -container-ip 172.18.0.3 -container-port 80 -use-listen-fd
             └─38416 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -container-ip 172.18.0.3 -container-port 80 -use-listen-fd
```

```
[benjamin@192 mi-proyecto]$ systemctl is-enabled docker
enabled
```

3. **Estructura del proyecto:** tree /home/benjamin/mi-proyecto mostrando el docker-compose.yaml, el Dockerfile, la carpeta src con index.php y el script DB.sql.

```
[benjamin@192 mi-proyecto]$  tree /home/benjamin/mi-proyecto
/home/benjamin/mi-proyecto
├── docker-compose.yaml
├── php-apache
│   ├── Dockerfile
│   └── src
│       └── index.php
└── SQL
    └── DB.sql

3 directories, 4 files
```

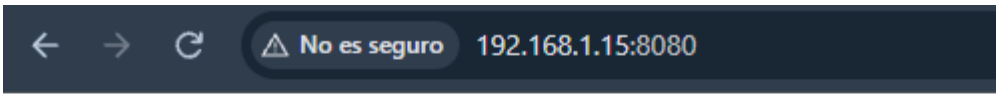
4. **Construcción y arranque:** docker compose up -d, con la salida de la construcción de la imagen y la creación de ambos contenedores con un solo comando.

```
[benjamin@192 mi-proyecto]$ sudo docker compose up -d
[+] up 2/2
 ✓ Container servidor_web   Running
 ✓ Container base_datos_sql Running
```

5. **Contenedores en ejecución:** docker compose ps mostrando servidor_web y base_datos_sql en estado Up, con los puertos 8080 y 6033 publicados.

```
[benjamin@192 mi-proyecto]$ sudo docker compose ps
NAME            IMAGE             COMMAND                  SERVICE   CREATED       STATUS         PORTS
base_datos_sql  mysql:8.0         "docker-entrypoint.s…"   db        4 hours ago   Up 7 minutes   33060/tcp, 0.0.0.0:6033->3306/tcp, [::]:6033->3306/tcp
servidor_web    mi-proyecto-web   "docker-php-entrypoi…"   web       4 hours ago   Up 7 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

6. **Aplicación web:** navegador accediendo a http://localhost:8080 y mostrando el mensaje de conexión exitosa a la base de datos desde index.php.



Conexión exitosa a la base de datos **mi_base**

7. **Base de datos inicializada:** ingreso a MySQL con docker exec -it base_datos_sql mysql -u usuario -p mi_base y ejecución de SHOW TABLES; para verificar que DB.sql se cargó automáticamente.

```
[benjamin@192 mi-proyecto]$ sudo docker exec -it base_datos_sql mysql -u usuario -p mi_base
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> SHOW TABLES;
+------------------+
| Tables_in_mi_base |
+------------------+
| usuarios         |
+------------------+
1 row in set (0.00 sec)
```

8. **Firewall:** sudo firewall-cmd --list-all mostrando el puerto 8080/tcp habilitado y sudo firewall-cmd --zone=trusted --list-all mostrando la interfaz docker0.

```
[benjamin@192 mi-proyecto]$ sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s3
  sources:
  services: cockpit dhcpv6-client ssh
  ports: 8080/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

```
[benjamin@192 mi-proyecto]$ sudo firewall-cmd --zone=trusted --list-all
trusted (active)
  target: ACCEPT
  icmp-block-inversion: no
  interfaces: docker0
  sources:
  services:
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

9. **Persistencia de los datos:** docker compose down seguido de docker compose up -d y docker volume ls, comprobando que el volumen datosDB sigue existiendo y que los datos se conservan.

```
[benjamin@192 mi-proyecto]$ sudo docker compose down
[+] down 3/3
 ✓ Container servidor_web      Removed
 ✓ Container base_datos_sql    Removed
 ✓ Network mi-proyecto_default Removed
[benjamin@192 mi-proyecto]$ sudo docker compose up -d
[+] up 3/3
 ✓ Network mi-proyecto_default Created
 ✓ Container base_datos_sql    Started
 ✓ Container servidor_web      Started
[benjamin@192 mi-proyecto]$ sudo docker volume ls
DRIVER    VOLUME NAME
local     datosDB
local     f81fb79a405eabd7383ff5ea0957226a0531213b10eb719c4a7daeabf27950d2
local     volumenPrueba
```

12. Administración del árbol de directorios y comandos del sistema operativo

En esta sección se amplía la documentación de los directorios y archivos que componen el proyecto, desde el punto de vista de la administración de sistemas operativos. Se toma como referencia el contenido real del archivo proyecto-roka.zip entregado junto con este documento. Las rutas mostradas son rutas Linux; deben adaptarse si el proyecto se instala en otra ubicación.

12.1 Ubicación del proyecto en el servidor

La carpeta raíz incluida en el ZIP se llama proyecto-roka. Para trabajar desde CentOS Stream 9, primero se debe ingresar al directorio donde se haya extraído el proyecto y comprobar la ubicación actual con pwd. En el documento anterior se utilizó /home/benjamin/mi-proyecto como ubicación de trabajo; esa ruta no coincide con el nombre de la carpeta raíz del ZIP y debe ajustarse en función de dónde se copie realmente.

pwd

ls -la

cd /ruta/donde/se/encuentra/proyecto-roka

pwd

El comando pwd muestra la ruta absoluta del directorio actual. ls -la lista también los archivos ocultos (por ejemplo, .htaccess), mientras que cd permite cambiar de directorio. Es importante verificar la ruta antes de ejecutar Docker Compose, ya que las rutas incorrectas impiden localizar el Dockerfile y los directorios que se montan dentro de los contenedores.

12.2 Árbol de directorios real del proyecto

El árbol siguiente resume la estructura relevante. Se omiten archivos de bibliotecas de terceros y recursos individuales para facilitar la lectura; no significa que esos archivos no formen parte del proyecto.

```
proyecto-roka/
├── docker-compose.yaml
└── php-apache/
    ├── Dockerfile
    └── src/
        ├── index.php
        └── sigsm/
            ├── .htaccess
            ├── api/
            │   ├── index.php
            │   ├── login.php
            │   ├── logout.php
            │   ├── usuarios.php
            │   └── ... (otros endpoints)
            ├── backend/
            │   ├── config/
            │   │   └── Database.php
            │   ├── database/
            │   │   ├── 01_sigsm.sql
            │   │   └── 02_usuarios_bd.sql
            │   ├── models/
            │   │   └── ... (modelos de datos)
            │   └── negocio/
```



```
│       └── ... (servicios y reglas)
├── docs/
├── frontend/
│   ├── css/
│   ├── img/
│   ├── js/
│   └── ... (páginas HTML)
├── uploads/
│   └── documentos/
└── README.md
```

La carpeta proyecto-roka/ es la raíz de la entrega de sistemas operativos. docker-compose.yaml define los servicios y recursos de Docker Compose. php-apache/ contiene el Dockerfile y el contenido que se monta en el servidor web. Dentro de src/sigsm/ se encuentra la aplicación SIGSM: api/ contiene los puntos de acceso, backend/ organiza la configuración, scripts SQL, modelos y lógica de negocio, frontend/ contiene los recursos de presentación y uploads/ almacena los documentos cargados.

La carpeta backend/database/ es especialmente importante para la inicialización de la base de datos. El compose incluido monta ese directorio en /docker-entrypoint-initdb.d del contenedor MySQL. Los scripts de esa carpeta se ejecutan durante la inicialización inicial de una instancia de base de datos vacía; no se vuelven a ejecutar automáticamente en cada reinicio si el volumen de datos ya contiene una base creada.

tree -L 5

tree permite visualizar jerárquicamente los directorios. Si el comando no está instalado, se puede utilizar find, que forma parte de las utilidades habituales del sistema. -L limita la profundidad mostrada.

12.3 Rutas absolutas y relativas en Docker Compose

Para que el despliegue sea portable, se recomienda utilizar rutas relativas al archivo docker-compose.yaml. En el servicio web, el contexto de construcción puede definirse como

./php-apache y el volumen del código como ./php-apache/src:/var/www/html:z. Para la inicialización de MySQL, la ruta puede definirse como ./php-apache/src/sigsm/backend/database:/docker-entrypoint-initdb.d:ro,z. La modificación de rutas debe realizarse antes de ejecutar docker compose up -d.

12.4 Configuración de acceso remoto al usuario de MySQL desde Docker

Debido a que la aplicación y el servidor de base de datos se ejecutan en contenedores Docker independientes, es necesario configurar correctamente los permisos de conexión del usuario utilizado por la aplicación.

Inicialmente, el usuario sigsm_app se encontraba asociado al host localhost, lo que restringía su autenticación a conexiones locales del servidor MySQL.

Para permitir que la aplicación pueda establecer conexiones desde otro contenedor dentro de la red Docker, se modifica el host asociado al usuario mediante el siguiente comando:

```
None
sudo docker exec -it base_datos_sql mysql -uroot -p -e "RENAME USER 'sigsm_app'@'localhost' TO 'sigsm_app'@'%';"
```

Esta modificación permite que el usuario pueda autenticarse desde diferentes hosts, siempre que exista conectividad con el servidor y se cumplan las condiciones de autenticación y seguridad configuradas.

Resultado esperado: la aplicación puede conectarse al servidor MySQL utilizando el usuario sigsm_app, sin quedar limitada a conexiones provenientes de localhost.

[ESPACIO PARA CAPTURA]

Figura X. Modificación del host asociado al usuario de MySQL mediante Docker.

13. Referencias técnicas y fecha de revisión

La información sobre el ciclo de vida de CentOS Stream se revisó el 08/11/2026 en las páginas oficiales del proyecto CentOS. El proyecto se publica el 31 de mayo de 2027 como fecha de fin de vida de CentOS Stream 9.

Proyecto CentOS - comparación y ciclo de vida: https://www.centos.org/cl-vs-cs/

Proyecto CentOS - descarga y versiones disponibles: https://www.centos.org/download/

Documentación CentOS Stream: https://docs.centos.org/